

JDBC 案例：条件查询和分页

今日内容介绍

- ◆ 案例：删除所选商品
- ◆ 案例：条件查询
- ◆ 案例：分页查询

今日内容学习目标

- ◆ 条件查询
- ◆ 分页查询

第1章 案例：分页查询

1.1 案例介绍

实际开发中，条件查询时非常常见的一种功能，上一个案例中我们已经完成。接着我们再来学习另一种也非常常见的功能：分页查询。

1.2 相关知识：sql 分页

格式：select * from 表 limit ?,?;

参数 1：开始索引 startIndex，默认值：0。必须是正数。

参数 2：每页显示个数 pageSize

例如：

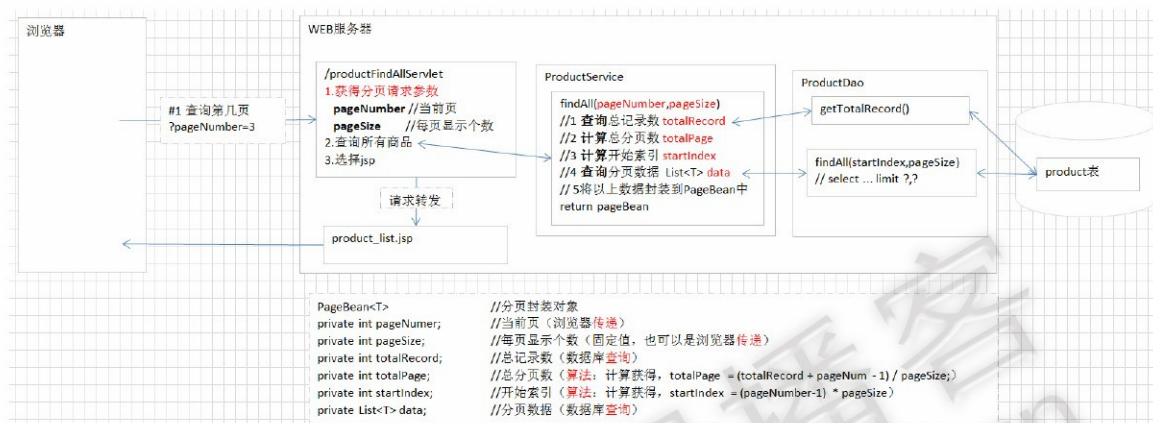
```
select * from products limit 0,5; #第一页，每页显示 5 条。
```

```

select * from products limit 5,5; #第二页，每页显示 5 条。
select * from products limit 10,5; #第三页，每页显示 5 条。
select * from products limit ?,5; #第 pageNumber 页，每页显示 5 条。
startIndex = (pageNumber - 1) * pageSize;

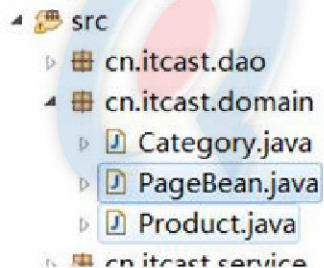
```

1.3 案例分析



1.4 案例实现

- 步骤 1: 编写 PageBean, 提供构造方法, 统一算法的编写。



```

public class PageBean<T> {
    private int pageNumber; //当前页 (浏览器传递)
    private int pageSize; //每页显示个数 (固定值, 也可以是浏览器传递)
    private int totalRecord; //总记录数 (数据库查询)
    private int totalPage; //总分页数
    private int startIndex; //开始索引
    private List<T> data; //分页数据 (数据库查询)

    public PageBean(int pageNumber, int pageSize, int totalRecord) {
        this.pageNumber = pageNumber;
        this.pageSize = pageSize;
        this.totalRecord = totalRecord;
    }
}

```

```
// 成员变量和局部变量重名，此处统一使用成员变量，及都需要添加“this.”
// #1 总分页数
this.totalPage = (this.totalRecord + this.pageNumber - 1) / this.pageSize;
// #2 开始索引
this.startIndex = (this.pageNumber-1) * pageSize;

}
```

- 步骤 2: 确定入口

http://localhost:8080/day14_page/productFindAllServlet?pageNumber=2

- 步骤 3: 编写 servlet，获得参数

```
ProductFindAllServlet.java
17 public void doGet(HttpServletRequest request, HttpServletResponse response)
18     throws ServletException, IOException {
19     //1 获得分页参数
20     String pageNumberStr = request.getParameter("pageNumber");
21     Integer pageNumber = 1; //处理没有传递参数的情况
22     try {
23         Integer.parseInt(pageNumberStr);
24     } catch (Exception e) {
25     }
26     Integer pageSize = 2; //固定值, Integer.parseInt(request.getParameter("pageSize"));
27
28     //2 通知service, 查询所有商品
29     ProductService productService = new ProductService();
30     PageBean<Product> pageBean = productService.findAll(pageNumber, pageSize);
31
32     //3 选择jsp
33     //3.1 将查询结果存放在request作用域
34     request.setAttribute("pageBean", pageBean); //将分页数据存放在request作用域
35     //3.2 请求转发
36     request.getRequestDispatcher("/admin/product/product_list.jsp").forward(request, response);
37
38 }
```

```
public class ProductFindAllServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //1 获得分页参数
        String pageNumberStr = request.getParameter("pageNumber");
        Integer pageNumber = 1;
        try {
            Integer.parseInt(pageNumberStr);
        } catch (Exception e) {
        }
        Integer pageSize = 2;
        //固定值, Integer.parseInt(request.getParameter("pageSize"));

        //2 通知 service, 查询所有商品
        ProductService productService = new ProductService();
        PageBean<Product> pageBean = productService.findAll(pageNumber, pageSize);
    }
}
```

```
//3 选择 jsp
//3.1 将查询结果存放在 request 作用域
request.setAttribute("pageBean", pageBean);
//3.2 请求转发
request.getRequestDispatcher("/admin/product/product_list.jsp")
    .forward(request, response);
}
```

- 步骤 4: 编写 service, 进行分页查询, 将所有数据封装到 PageBean 中。

```
/**
 * 查询所有, 分页
 * @return
 */
public PageBean<Product> findAll(int pageNumber, int pageSize){

    //1 查询总记录数
    int totalRecord = productDao.getTotalRecord();
    //2 总分页数、开始索引, 已经在 PageBean 中处理了
    PageBean<Product> pageBean = new PageBean<Product>(pageNumber, pageSize,
totalRecord);

    //3 查询分页数据
    List<Product> data =
        productDao.findAll(pageBean.getStartIndex(), pageBean.getPageSize());
    // * 将分页数据封装 PageBean 中
    pageBean.setData(data);

    //4 返回封装好的数据
    return pageBean;
}
```

- 步骤 5: 编写 dao, 查询总记录数和分页数据

```
/**
 * 查询总记录数
 * @return
 */
public int getTotalRecord() {
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        String sql = "select count(*) from product";
        Object[] params = {};
        Long numLong = (Long) queryRunner.query(sql, new ScalarHandler(), params);
        return numLong.intValue();
    } catch (Exception e) {
```

```

        throw new RuntimeException(e);
    }
}

/**
 * 分页查询所有
 * @return
 */
public List<Product> findAll(int startIndex, int pageSize) {
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        String sql = "select * from product limit ?,?";
        Object[] params = {startIndex, pageSize};
        return queryRunner.query(sql, new BeanListHandler<Product>(Product.class),
params);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
}

```

ProductDao.java

```

47  /**
48  * 分页查询所有
49  * @return
50  */
51  public List<Product> findAll(int startIndex, int pageSize) {
52      try {
53          QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
54          String sql = "select * from product limit ?,?";
55          Object[] params = {startIndex, pageSize};
56          return queryRunner.query(sql, new BeanListHandler<Product>(Product.class), params);
57      } catch (Exception e) {
58          throw new RuntimeException(e);
59      }
60  }

```

- 步骤 6: 修改 jsp 页面，显示分页数据

位置: /day14_page/WebContent/admin/product/product_list.jsp

product_list.jsp

```

115  <!-- 列表项 start -->
116  <c:forEach items="${pageBean.data}" var="product">
117      <tr onmouseover="this.style.backgroundColor = '#F5FAFE'"
118          onmouseout="this.style.backgroundColor = '#fff';">

```

- 步骤 7: 显示分页信息

```

第${ pageBean.pageNumber }/${ pageBean.totalPage }页       
总记录数:${pageBean.totalRecord }     每页显示: ${ pageBean.pageSize }       
<c:if test="${ pageBean.pageNumber gt 1 }">
    <a

```

```
href="${ pageContext.request.contextPath }/productListPageServlet?pageNumber=1">[ 首
页]</a> | &nbsp;
    <a
href="${ pageContext.request.contextPath }/productListPageServlet?pageNumber=${page
Bean.pageNumber-1}">[上一页]</a>|&nbsp;
    </c:if>
    &nbsp;
    <c:forEach var="i" begin="1" end="${pageBean.totalPage }">
        <c:if test="${ pageBean.pageNumber == i }">
            ${ i }
        </c:if>
        <c:if test="${ pageBean.pageNumber != i }">
            <a
href="${ pageContext.request.contextPath }/productListPageServlet?pageNumber=${ i}"
>${ i }</a>
        </c:if>
    </c:forEach>
    &nbsp;
    <c:if test="${ pageBean.pageNumber lt pageBean.totalPage }">
        <a
href="${ pageContext.request.contextPath }/productListPageServlet?pageNumber=${page
Bean.pageNumber+1}">[下一页]</a>|&nbsp;
        <a
href="${ pageContext.request.contextPath }/productListPageServlet?pageNumber=${page
Bean.totalPage}">[尾页]</a>
    </c:if>
```

第2章 案例：条件查询

2.1 案例介绍

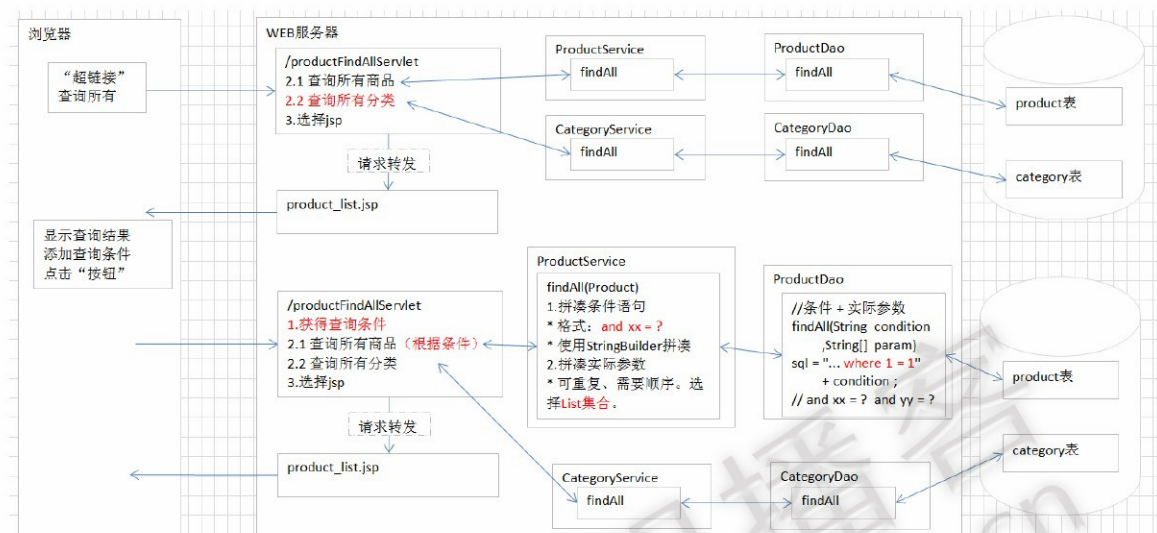
我们已经完成了“查询所有商品”功能，如果我们希望从所有数据中筛选出需要的部分数据，会发现通过肉眼比较困难，且没有必要。在开发中，我们采用“条件查询”来解决此类问题。

查询条件	
分类	
商品名称:	
查询 重置	

2.2 案例分析

“条件查询”和“查询所有”，两个功能非常相似，不同出就在条件的筛选上，通过页面将分类和商品名称传递给服务器，服务器获得查询条件，然后通知 service 拼凑 sql 语句进行具体查询。

我们发现如果要完成“条件查询”，必须在之前的“查询所有”时，查询所有的分类，从而可以进行条件的选择。



2.3 案例实现

- 步骤 2: 确定 JavaBean，在商品 Product 对象中，使用的“category_id”字段存放分类 id。

```

Product.java
14
15 private String pid;
16 private String pname;
17 private Double price;
18 private String pimage;
19 private String description;
20 private String category_id;

```

- 步骤 2: 修改 Servlet，查询所有时，查询所有分类（条件查询使用）
修改: `/day14_condition/src/cn/itcast/web/servlet/ProductFindAllServlet.java`

```
ProductFindAllServlet.java
18
19 public void doGet(HttpServletRequest request, HttpServletResponse response)
20     throws ServletException, IOException {
21     //1
22
23     //2.1 通知 service, 查询所有商品
24     ProductService productService = new ProductService();
25     List<Product> allProduct = productService.findAll();
26     //2.2 通过分类 service, 查询所有分类
27     CategoryService categoryService = new CategoryService();
28     List<Category> allCategory = categoryService.findAll();
29
30     //3 选择jsp
31     //3.1 将查询结果, 所有商品, 存放在 request 作用域
32     request.setAttribute("allProduct", allProduct);
33     //3.2 将查询结果, 所有分类, 存放在 request 作用域
34     request.setAttribute("allCategory", allCategory);
35     //3.3 请求转发
36     request.getRequestDispatcher("/admin/product/product_list.jsp").forward(request, response);
37
38 }
```

```
public class ProductFindAllServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //1

        //2.1 通知 service, 查询所有商品
        ProductService productService = new ProductService();
        List<Product> allProduct = productService.findAll();
        //2.2 通过分类 service, 查询所有分类
        CategoryService categoryService = new CategoryService();
        List<Category> allCategory = categoryService.findAll();

        //3 选择jsp
        //3.1 将查询结果, 所有商品, 存放在 request 作用域
        request.setAttribute("allProduct", allProduct);
        //3.2 将查询结果, 所有分类, 存放在 request 作用域
        request.setAttribute("allCategory", allCategory);
        //3.3 请求转发
        request.getRequestDispatcher("/admin/product/product_list.jsp")
            .forward(request, response);

    }
}
```

- 步骤 3: 修改 product.jsp, 显示所有分类

修改: /day14_condition/WebContent/admin/product/product_list.jsp

```

product_list.jsp
38 class="ta_01" bgColor="#ffffff" style="width: 30%">
39 <select name="" class="form-control">
40     <option value="">--请选择分类--</option>
41     <c:forEach items="${allCategory}" var="category">
42         <option value="${category.cid}">${category.cname}</option>
43     </c:forEach>
44 </select>

```

```

<c:forEach items="${allCategory}" var="category">
    <option value="${category.cid}">${category.cname}</option>
</c:forEach>

```

分类	--请选择分类-- --请选择分类-- 服装 测试 美食
----	--

- 步骤 4: 修改 product.jsp, 完善表单。概览如下:

```

<form action="${pageContext.request.contextPath}/productFindAllServlet" method="post">
    分类 <select name="category_id" class="form-control">
    商品名称: <input type="text" name="pname" size="15" class="form-control"/>
    <button type="submit" value="查询" class="button_view">查询</button>

```

- 步骤 5: 完善 servlet, 获得查询条件, 并封装到 javabean, 然后传递给 service
位置: /day14_condition/src/cn/itcast/web/servlet/ProductFindAllServlet.java

```

ProductFindAllServlet.java
16
17 public class ProductFindAllServlet extends HttpServlet {
18     private static final long serialVersionUID = 1L;
19
20     public void doGet(HttpServletRequest request, HttpServletResponse response)
21         throws ServletException, IOException {
22         //1 获得请求参数
23         Product product = MyBeanUtils.populate(Product.class, request.getParameterMap());
24
25         //2.1 通知 service, 查询所有商品
26         ProductService productService = new ProductService();
27         List<Product> allProduct = productService.findAll(product);
28         //2.2 通过分类 service, 查询所有分类
29         CategoryService categoryService = new CategoryService();
30         List<Category> allCategory = categoryService.findAll();

```

此时 ProductService 的 findAll(product) 不存在所以抛异常。

```

public class ProductFindAllServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //1 获得请求参数
        Product product = MyBeanUtils.populate(Product.class, request.getParameterMap());

        //2.1 通知 service, 查询所有商品
        ProductService productService = new ProductService();
        List<Product> allProduct = productService.findAll(product);

```

//2.2 ...省略

- 步骤 6: 完善 service, 接收条件, 并拼凑 sql 查询条件。

```
/**
 * 查询所有(含条件)
 * @return
 */
public List<Product> findAll(Product product){

    //1.1 拼凑 sql 语句, 格式: and 属性 符号 ?
    StringBuilder builder = new StringBuilder();
    //1.2 拼凑实际参数。实际参数可重复、参数需要顺序, 所以选择 List 集合
    List<Object> paramsList = new ArrayList<Object>();

    //2.1 分类不为空, 添加分类条件
    if(product.getCategory_id() != null && !"".equals(product.getCategory_id())){
        builder.append(" and category_id = ? ");
        paramsList.add(product.getCategory_id());
    }

    //2.2 商品名不为空, 添加商品名条件
    if(product.getPname() != null && !"".equals(product.getPname())){
        builder.append(" and pname like ? ");
        //商品名需要模糊查询即可, 需要手动的添加%
        paramsList.add("%"+product.getPname()+"%");
    }

    //3 转换成需要的类型
    String condition = builder.toString();
    Object[] params = paramsList.toArray();

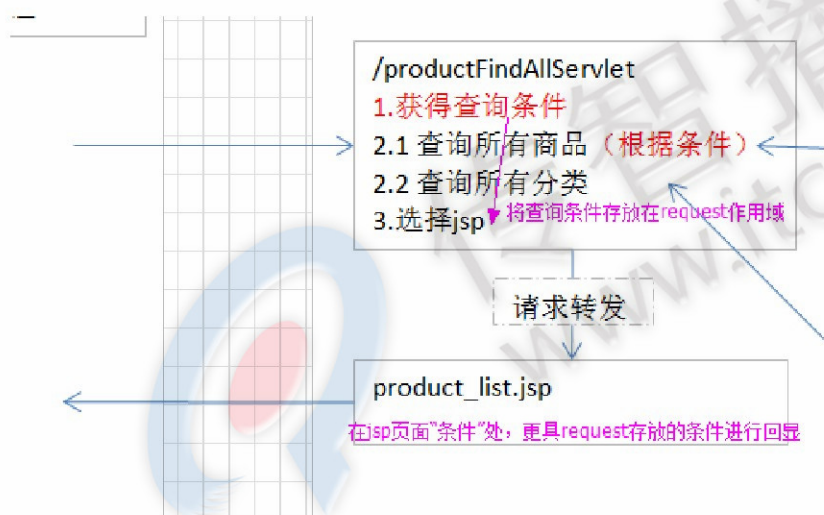
    return productDao.findAll(condition , params);
}
```

- 步骤 7: 完善 dao, 使用条件和参数

```
ProductDao.java
29 /**
30  * 查询所有
31  * @return
32  */
33 public List<Product> findAll(String condition , Object[] params){
34     try {
35         QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
36         String sql = "select * from product where 1=1 " + condition;
37         return queryRunner.query(sql, new BeanListHandler<Product>(Product.class), params);
38     } catch (Exception e) {
39         throw new RuntimeException(e);
40     }
41 }
42 /**
```

```
* 查询所有
* @return
*/
public List<Product> findAll(String condition , Object[] params){
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        String sql = "select * from product where 1=1 " + condition;
        return queryRunner.query(sql, new BeanListHandler<Product>(Product.class),
params);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

2.4 查询条件回显



- 步骤 1: 修改 servlet，将封装好的查询条件，添加到 request 作用域
`/day14_condition/src/cn/itcast/web/servlet/ProductFindAllServlet.java`

```
ProductFindAllServlet.java
19
20 public void doGet(HttpServletRequest request, HttpServletResponse response)
21     throws ServletException, IOException {
22     //1 获得请求参数
23     Product product = MyBeanUtils.populate(Product.class, request.getParameterMap());
24
25     //2.1 通知 service, 查询所有商品
26     ProductService productService = new ProductService();
27     List<Product> allProduct = productService.findAll(product);
28     //2.2 通过分类 service, 查询所有分类
29     CategoryService categoryService = new CategoryService();
30     List<Category> allCategory = categoryService.findAll();
31
32     //3 选择 jsp
33     //3.1 将查询结果, 所有商品, 存放在 request 作用域
34     request.setAttribute("allProduct", allProduct);
35     //3.2 将查询结果, 所有分类, 存放在 request 作用域
36     request.setAttribute("allCategory", allCategory);
37     //3.3 存放条件
38     request.setAttribute("product", product);
39     //3.4 请求转发
40     request.getRequestDispatcher("/admin/product/product_list.jsp").forward(request, response);
41 }
42 }
```

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    //1 获得请求参数
    Product product = MyBeanUtils.populate(Product.class,
request.getParameterMap());

    //2.1 通知 service, 查询所有商品
    ProductService productService = new ProductService();
    List<Product> allProduct = productService.findAll(product);
    //2.2 通过分类 service, 查询所有分类
    CategoryService categoryService = new CategoryService();
    List<Category> allCategory = categoryService.findAll();

    //3 选择 jsp
    //3.1 将查询结果, 所有商品, 存放在 request 作用域
    request.setAttribute("allProduct", allProduct);
    //3.2 将查询结果, 所有分类, 存放在 request 作用域
    request.setAttribute("allCategory", allCategory);
    //3.3 存放条件
    request.setAttribute("product", product);
    //3.4 请求转发
    request.getRequestDispatcher("/admin/product/product_list.jsp").forward(request, response);

}
```

- 步骤 2: 使用“条件 product”进行标签回显
/day14_condition/WebContent/admin/product/product_list.jsp

分类

```
<c:forEach items="${allCategory}" var="category">
```

```
  <option value="${category.cid}"
```

```
    ${product.category_id == category.cid ? "selected='selected'" : ""} >
```

```
    ${category.cname}
```

```
  </option>
```

```
</c:forEach>
```

商品名称:

```
<input type="text" name="pname" size="15" value="${product.pname}"
```

```
class="form-control"/>
```

查询条件								
分类	服装	商品名称:	1	查询	重置			
商品列表								
删除选中	预览图	商品分类	商品名称	商品价格	描述	编辑	查看	删除
☐		1	1	1.0	1			

第3章 总结